

---

# Exact Reduction of Huge Action Spaces in General Reinforcement Learning

SULTAN J. MAJEED<sup>†</sup>

*Research School of Computer Science, ANU*

[sultan.pk](mailto:sultan.pk)

MARCUS HUTTER

*Google DeepMind & Research School of Computer Science, ANU*

[hutter1.net](mailto:hutter1.net)

---

## Abstract

The reinforcement learning (RL) framework formalizes the notion of learning with interactions. Many real-world problems have large state-spaces and/or action-spaces such as in Go, StarCraft, protein folding, and robotics or are non-Markovian, which cause significant challenges to RL algorithms. In this work we address the large action-space problem by sequentializing actions, which can reduce the action-space size significantly, even down to two actions at the expense of an increased planning horizon. We provide explicit and exact constructions and equivalence proofs for all quantities of interest for arbitrary history-based processes. In the case of MDPs, this could help RL algorithms that bootstrap. In this work we show how action-binarization in the non-MDP case can significantly improve Extreme State Aggregation (ESA) bounds. ESA allows casting any (non-MDP, non-ergodic, history-based) RL problem into a fixed-sized non-Markovian state-space with the help of a surrogate Markovian process. On the upside, ESA enjoys similar optimality guarantees as Markovian models do. But a downside is that the size of the aggregated state-space becomes exponential in the size of the action-space. In this work, we patch this issue by binarizing the action-space. We provide an upper bound on the number of states of this binarized ESA that is logarithmic in the original action-space size, a double-exponential improvement.

## Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>1</b>  |
| <b>2</b> | <b>Notation</b>                            | <b>4</b>  |
| <b>3</b> | <b>Problem Setup</b>                       | <b>4</b>  |
| 3.1      | General Reinforcement Learning . . . . .   | 5         |
| 3.2      | Sequential Decisions . . . . .             | 6         |
| <b>4</b> | <b>Sequentialized Processes and Values</b> | <b>9</b>  |
| <b>5</b> | <b>Extreme State Aggregation</b>           | <b>13</b> |
| <b>6</b> | <b>Conclusion &amp; Outlook</b>            | <b>16</b> |

## 1 Introduction

The reinforcement learning (RL) setting can be described by an agent-environment interaction [? ]. The agent  $\Pi$  has an action-space  $\mathcal{A}$  to choose its actions from while the environment  $P$  reacts to the action by dispensing an observation and a reward from the sets  $\mathcal{O}$  and  $\mathcal{R} \subseteq \mathbb{R}$ ,

---

<sup>†</sup>The contact author.

respectively, see Figure 1. For simplicity, we assume that these sets are finite and hence the rewards are bounded. Even with these restrictions, the problem of RL does not trivialize, i.e. the agent can not learn the optimal behavior without further structure. Under a suitable definition of the “state” of environment, the resultant set of states might be huge or even infinite [? ].

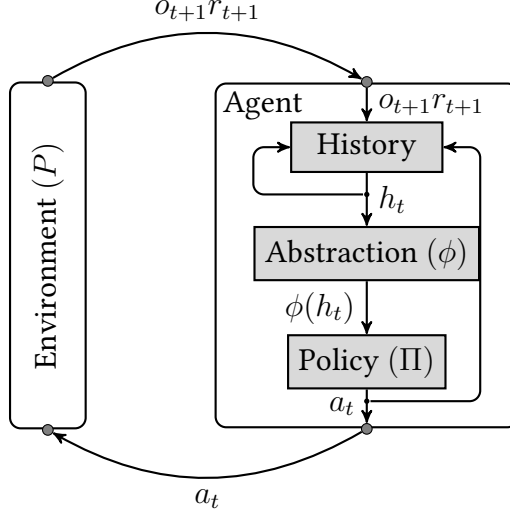


Figure 1: The agent-environment interaction.

The problem of RL is plagued with the curse of dimensionality. The sizes of an appropriately defined set of states<sup>1</sup> and action-space play an important role in the choice of algorithms, architectures and solution techniques used to solve the task [? ].

It is usually required to produce a relatively small set of states to make the problem tractable [? ]. There are many ways to achieve such approximations/abstractions, e.g. state aggregation [? ? ? ], homomorphism [? ], linear function approximation [? ], or neural networks [? ] just to name a few. The usual assumption for such abstractions is that they try to produce a Markovian representation of the environment, which is known as a Markov Decision Process (MDP) [? ]. In an MDP the most recent (abstract) observation is sufficient to predict *any* future event<sup>2</sup>, but not all events are equally valuable. Some events might not lead to a high rewarding state and/or some distinctions are not really necessary to perform well [? ]. For example, the agent might end up experiencing two completely different streams of observations with the same reward structure. An algorithm which tries to produce a Markovian representation would try to make this “unnecessary” distinction.

A lot can be achieved in terms of the representation power if an algorithm only makes “useful” distinctions, i.e. the distinctions (or states) which respect the reward structure [? ? ? ]. In some cases, such “useful” but non-Markovian abstractions reduce the effective state-space dramatically. Usually, a smaller state-space facilitates faster learning [? ? ].

The usual methods of state or action-space reductions either (1) reduce the problem to a fixed size where the quality of reduction deteriorates as the original problem becomes more complicated, or (2) provide a problem-specific reduction which usually grows, albeit much slower, as the original problem grows [? ]. In Markovian abstractions the size of the state-space grows with the size of the observation and reward spaces. For example, if an MDP

<sup>1</sup>We prefer not to call the set of observations  $\mathcal{O}$  the set of states. This set becomes a set of states under strong assumptions, see Section 5 for more details on this distinction.

<sup>2</sup>An event is any set of action-observation-reward sequences.

abstraction produces states from some low-resolution images then we need more states to handle high-resolution versions of the input images because the high-resolution images need a bigger transition matrix to predict the next image. However, it is perfectly plausible that the increased resolution might not be “useful” to achieve better rewards.

To the best of our knowledge, extreme state aggregation (ESA), a non-MDP abstraction framework, is the only method which provides a provable upper bound on the size of required state-space uniformly<sup>3</sup> for all problems [? ]. However, a *downside* of ESA is that the size of the aggregated state-space is *exponential* in the size of the action-space, see [Theorem 5.2](#). In this paper, we move the research further in this direction. We provide a variant of ESA that can help provide much more compact representations as compared to MDP abstractions. Our approach improves the key upper bound on the size of the state-space in the original ESA framework.

The key trick to achieve this improvement is to sequentialize the actions. Often  $\mathcal{A}$  already has a natural vector structure  $\mathcal{B}^d$ , e.g. real valued activators in robotics ( $\mathcal{B} = \mathbb{R}$ ) or (padded) words ( $\mathcal{B} = \{a, \dots, z, \_ \}$ ), or more generally  $\mathcal{B}_1 \times \dots \times \mathcal{B}_d$ , where  $\mathcal{B}$  denotes a finite set of *decision symbols*. In this case, sequentialization is natural, but one may further want to binarize  $\mathcal{B}$  to  $\mathbb{B}^d$  esp. for ESA ([Theorem 5.3](#)). If actions are (converted to)  $\mathcal{B}$ -ary strings, the RL agent could execute the action “bits” sequentially with fictitious dummy observations in-between.

The example in [Figure 2](#) provides a naive way of *sequentializing* the actions in an MDP. Apparently, it might seem that such sequentialization of the action-space would be of no help, as the state-space would blow up, and it is simply shifting the problem from the actions to the states. However, we prove that this can be avoided. Most importantly, the universal upper bound on the effective state-space of ESA remains valid. Our scheme of sequentializing the actions achieves a *double exponentially* improved bound; compare [Theorem 5.3](#) with [Theorem 5.2](#).

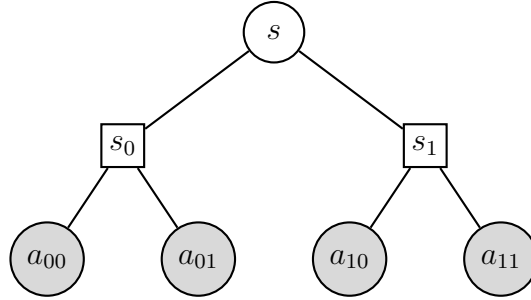


Figure 2: A simple sequentialization example in an MDP. To see how the actions sequentialized, consider an agent which has to choose among four alternatives, e.g.  $\mathcal{A} = \{a_{00}, a_{01}, a_{10}, a_{11}\}$ . Let the agent receive a state signal  $s$  from the environment. It first decides between a partition of actions, say two actions each,  $\{a_{00}, a_{01}\}$  and  $\{a_{10}, a_{11}\}$ . *After* it has decided on the bifurcation, the *extended* state becomes  $s_x$ , where  $x$  is the decision of the first stage. Now the agent *on this extended state*  $s_x$  makes its second decision to choose from the *short-listed* set of actions. This way, the agent only selects among *two* alternatives at each stage by *tripling* the effective state-space.

Along the way, we also establish some other key results, which are interesting and useful on their own. We provide explicit and exact constructions and equivalence proofs for all quantities of interest ([Section 4](#)) for arbitrary history-based processes, which are then used

<sup>3</sup>Which depends only on the size of the action-space, discount factor and the optimality gap.

to double-exponentially improve the previous ESA bound (Theorem 5.3). In the special case of MDPs, we show that through a sequentialized scheme (of augmenting observations with partial decision vectors) the resultant “sequentialized process” preserves the Markov property (Theorem 4.2), which should help RL algorithms that bootstrap, though demonstrating or proving this is left for future work. Moreover in Theorem 4.6, we prove that the stipulated sequentialization scheme preserves near-optimality, i.e. a near-optimal policy of the sequentialized process is also near-optimal in the original process.

The rest of the paper is organized as follows. Section 2 puts down the necessary notation. In Section 3, we formally set up the problem. A framework to sequentialize actions is provided in Section 4, along with other key results (Theorems 4.2 and 4.6). In Section 5 we combine our sequentialization framework with ESA to improve the upper bound on the size of the state-space (Theorem 5.3). We conclude the paper in Section 6 with an outlook.

## 2 Notation

This paper is notation heavy, but we use a consistent notation through out. The set of natural numbers is  $\mathbb{N} := \{1, 2, \dots\}$ ,  $\mathbb{B} := \{0, 1\}$  is a set of binary symbols, and  $\mathbb{R}$  is the set of reals. We denote by  $\Delta(X)$  the set of probability distributions over any set  $X$ . The concatenation of two objects (or strings) is expressed through juxtaposition, e.g.  $xy$  is a concatenation of  $x$  and  $y$ . We express a finite string with boldface, e.g.  $\mathbf{x} = x_1x_2 \dots x_{|\mathbf{x}|}$  where  $|\cdot|$  is used to denote the length or cardinality of the object. The individual members of a string or a vector may be accessed as  $\mathbf{x}_i = x_i$  for any  $i \leq |\mathbf{x}|$ . A substring of length  $i \leq |\mathbf{x}|$  is denoted as  $\mathbf{x}_{\leq i} = x_1x_2 \dots x_i$  and  $\mathbf{x}_{< i} = x_1x_2 \dots x_{i-1}$ . We interchangeably use the same notation for vectors and strings, e.g.  $\mathbf{x} \in \mathcal{B}^d$  is a  $d$ -dimensional  $\mathcal{B}$ -ary decision vector which may also be expressed as a string. This choice simplifies the notation and saves redundant variables. If a variable is time-indexed, we express the continuation of the variable with a prime on it, e.g. if  $x := x_t$  then  $x' := x_{t+1}$  where  $:=$  denotes equality by definition. A small scalar value (usually the error tolerance) is denoted by  $\varepsilon > 0$ . A different member of the same set is expressed with a dot on it, e.g.  $x, \dot{x} \in \mathcal{B}$ . We express the fact of  $\mathbf{x}$  being a prefix of  $\mathbf{y}$  by  $\mathbf{x} \sqsubseteq \mathbf{y}$  or  $\mathbf{y} \sqsupseteq \mathbf{x}$ . Moreover,  $\overline{\mathbf{x}\mathbf{y}}$  represents a vector that is point-wise joined, i.e.  $\overline{\mathbf{x}\mathbf{y}}_i := \mathbf{x}_i\mathbf{y}_i$ .

## 3 Problem Setup

This section provides the formal foundations for us to build up to the main result of this work. We formulate the problem of RL from the ground up without starting from the usual Markovian assumption [?]. We formalize a history-based RL setup. After formalizing the (general) RL problem, we set up the scheme of sequentializing the decision-making process to reduce the effective action-space for the agent. Especially for our main result about ESA, we sequentialize the action-space to *binary* decisions.

Although this work assumes very little about the RL problem, we assume that the size of the action-space is finite and  $|\mathcal{A}| = |\mathcal{B}|^d$  for some  $d \in \mathbb{N}$ . The latter assumption is not restrictive, as we can extend the set of actions by repeating some of the actions. It is important to note that these repeated actions should be labeled distinctly. This way we can have a bijection between the original (extended) action-space and the sequentialized one. For example, let an action set be  $\{a_1, a_2, a_3, a_4, a_5\}$ . One possible extended set, with repetition, for  $|\mathcal{B}| = 2$  and  $d = 3$  is  $\mathcal{A} := \{a_1, a_2, a_3, a_4, a_5, a_{5_1}, a_{5_2}, a_{5_3}\}$ . Where, the actions  $a_{5_i}$  for  $i \leq 3$  are functionally the same as  $a_5$ , i.e. taking  $a_5$  or any  $a_{5_i}$  action has the same effect, but they are labeled

distinctly.

Note that continuous action-spaces could be approximately sequentialized/binarized by using the binary expansion of reals to some desired precision, say  $\delta$ . Our main bound will only depend logarithmically on  $\delta$ .

### 3.1 General Reinforcement Learning

We consider a general reinforcement learning (GRL) setup where the agent keeps the complete history of interaction [? ]. The (infinite<sup>4</sup>) interaction produces an infinite history. Recall that  $\mathcal{O}$ ,  $\mathcal{R}$  and  $\mathcal{A}$  represent some *finite* sets of observations, rewards and actions, respectively. This also implies that the rewards are bounded. The set of all finite histories<sup>5</sup> is denoted by

$$\mathcal{H} := \bigcup_{t=1}^{\infty} \underbrace{\mathcal{O} \times \mathcal{R} \times \mathcal{A} \times \dots \times \mathcal{O} \times \mathcal{R} \times \mathcal{A}}_{(t-1)\text{-step interactions}} \times \mathcal{O} \times \mathcal{R} \quad (1)$$

which is used to express most of the quantities in our setup, e.g. the environment, agent, and value functions. Note that the history set  $\mathcal{H}$  does not contain the empty history. This is a design choice we make to be consistent with the standard RL setup [? ] where the initial state (in our case the initial observation and reward) is chosen by some “initial” distribution.

Formally, the environment  $P$  is a (conditional) probability function such that  $P : \mathcal{H} \times \mathcal{A} \rightarrow \Delta(\mathcal{O} \times \mathcal{R})$ . Similarly, the agent is also expressible as a (conditional) distribution on the action-space, i.e.  $\Pi : \mathcal{H} \rightarrow \Delta(\mathcal{A})$ . For a fixed policy  $\Pi$  the expected discounted future sum of rewards is the value of the policy. At any history  $h \in \mathcal{H}$  and action  $a \in \mathcal{A}$  the action-value function (or Q-function) is expressed as

$$Q^{\Pi}(h, a) := \sum_{o'r'} P(o'r'|ha) (r' + \gamma V^{\Pi}(hao'r')) \quad (2)$$

where  $V^{\Pi}(h) := \sum_a Q^{\Pi}(h, a) \Pi(a|h)$  is the value function of  $\Pi$  and  $0 \leq \gamma < 1$  is the discount factor. Equation (2) is known as the Bellman equation (BE). The optimal behavior (or policy) is the one which achieves the maximum value for all histories, i.e.  $\Pi^*(h) := \arg \max_{\Pi} V^{\Pi}(h)$ . The optimal value (and action-value) functions,  $V^* := V^{\Pi^*}$  and  $Q^* := Q^{\Pi^*}$ , of this optimal policy satisfy the following optimal Bellman equation (OBE) [? ? ].

$$Q^*(h, a) := \sum_{o'r'} P(o'r'|ha) (r' + \gamma V^*(hao'r')) \quad (3)$$

where  $V^*(h) := \max_a Q^*(h, a)$ . The agent defined in this sub-section works with the original action-space  $\mathcal{A}$  and keeps the histories from  $\mathcal{H}$ . In the next sub-section, we formulate another agent which only works in the “sequentialized” action-space, i.e. it takes decisions in a sequence of  $\mathcal{B}$ -ary choices, and responds *only* to the histories generated by this  $\mathcal{B}$ -ary interaction, see Figure 3. In the extreme case, this agent may only take binary decisions by sequentializing the action-space to binary sequences, i.e.  $\mathcal{B} = \mathbb{B}$ .

<sup>4</sup>For simplicity, we assume the interaction never stops. We do not consider the case where the agent or the environment can stop responding. It complicates the modeling beyond the scope of this work.

<sup>5</sup>Note that this set (of underlying “state” space) is (countably) infinite.

### 3.2 Sequential Decisions

We want to transform the action-space into a sequence of  $\mathcal{B}$ -ary decision code words, which are decided sequentially. To map the actions between the original action-space and the sequentialized decision-space, we define a pair of encoder and decoder functions. Let  $C$  be any encoding function that maps the actions to a  $\mathcal{B}$ -ary decision code of length  $d$ , i.e.  $C : \mathcal{A} \rightarrow \mathcal{B}^d$ . A decoder function  $D : \mathcal{B}^d \rightarrow \mathcal{A}$  sends the  $\mathcal{B}$ -ary decision sequences generated by  $C$  back to the actions in the (original) action-space. In this work, the choices of  $C$  and  $D$  do not matter<sup>6</sup> as long as they are bijections such that  $D(C(\mathcal{A})) = \mathcal{A}$ .

This sequentialization of the action-space changes the interaction history. The generated histories are no longer members of  $\mathcal{H}$ . The goal of this paper is to argue that an agent can still work with the sequentialized histories only. The agent can plan, learn and interact with the environment using  $\mathcal{B}$ -ary actions and keeping sequentialized histories. Hence, the agent can be agnostic to the original action-space and with the state provided through an appropriate abstraction it can only take  $\mathcal{B}$ -ary decisions at every time step, see Figure 3.

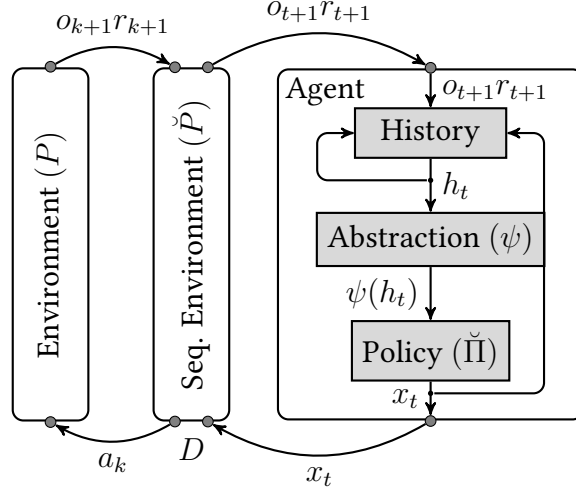


Figure 3: The agent-environment interaction through the sequentialization scheme. Note that the sequentialized-environment block (or a  $\mathcal{B}$ -ary “mock”) manages two different time-scales  $t$  and  $k$ . It is simply a buffer block which knows (de)coders  $C$  and  $D$  (see text for details). It buffers the input  $\mathcal{B}$ -ary actions and dispatches the buffered observation and reward. Once a complete  $\mathcal{B}$ -ary decision sequence is produced by the agent the  $\mathcal{B}$ -ary mock decodes the encoded actions to the original environment to continue the interaction loop. We can consider this sequentialized environment as a “middle layer” between the agent and the original environment.

We construct a history transformation function which maps the original histories from  $\mathcal{H}$  to some sequentialized histories in  $\check{\mathcal{H}}$ , where

$$\check{\mathcal{H}} := \bigcup_{t=1}^{\infty} \underbrace{\mathcal{O} \times \mathcal{R} \times \mathcal{B} \times \dots \times \mathcal{O} \times \mathcal{R} \times \mathcal{B}}_{(t-1)\text{-step interactions}} \times \mathcal{O} \times \mathcal{R} \quad (4)$$

It is worth noting that  $\check{\mathcal{H}}$  does not (directly) contain any information about  $\mathcal{A}$ , cf. Equation (1). The agent experiencing histories from this set would not be aware of  $\mathcal{A}$ .

<sup>6</sup>The choice could matter in practical implementation of such agents. For example, a clever choice of such functions might produce sparse  $\mathcal{B}$ -ary decision sequences for the optimal actions, hence it may facilitate in learning such optimal  $\mathcal{B}$ -ary decision sequences.

**Definition 3.1** (History transformation function). A history transformation function is expressed with  $g : \mathcal{H} \rightarrow \check{\mathcal{H}}$ . The map is recursively defined for any history  $h$ , action  $a$ , next observation  $o'$  and next reward  $r'$  as

$$g(hao'r') := g(h)\mathbf{x}_1or_\perp\mathbf{x}_2or_\perp\ldots\mathbf{x}_d o'r' \text{ and } g(e) := e \quad (5)$$

where  $\mathbf{x} := C(a)$ ,  $o$  is the last observation of the history  $h$ ,  $e$  denotes the “initial” history<sup>7</sup>, and  $r_\perp$  is any fixed real-value. In this work, we assume<sup>8</sup> that  $r_\perp \in \mathcal{R}$  and  $r_\perp = 0$ .

In the above construction, we chose to repeat the last observation  $o$  in between the real interactions with the environment. This is not the only possible choice, we can choose a *dummy* observation  $o_\perp \in \mathcal{O}$  instead without affecting the claims. For brevity, we define  $\mathbf{o}$  and  $\mathbf{r}_\perp$  as  $d$ -dimensional constant vectors of  $o$  and  $r_\perp$ , respectively. These vectors are then “welded” with  $\mathbf{x}$  to succinctly replace  $x_1or_\perp\ldots x_ior_\perp$  with  $\overline{\mathbf{x}\mathbf{o}\mathbf{r}_\perp}_{\perp \leq i}$ . Note that we do not sequentialize the observations. It can be done, but we believe it is not useful in any way.

However, if the original process  $P$  is an MDP, i.e. the most recent observation is the state of  $P$ , then there is another interesting option possible for  $o_\perp$ : extend the observation space  $\mathcal{O}$  with  $\mathcal{O} \times \cup_{i=0}^{d-1} \mathcal{B}^i =: \tilde{\mathcal{O}}$ , and let the  $\mathcal{B}$ -ary mock dispatch an appropriate observation at every partial  $\mathcal{B}$ -ary decision vector  $\mathbf{x}_{<i}$  as:

$$\tilde{o}_\perp := (o, \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{i-1}) \in \tilde{\mathcal{O}} \quad (6)$$

It is not hard to show that with this sequentialization scheme the resultant sequentialized decision process is also an MDP over  $\tilde{\mathcal{O}}$ , see [Theorem 4.2](#). By doing so, we end up with a state-space of size  $|\tilde{\mathcal{O}}| = |\mathcal{O}|(|\mathcal{A}| - 1) \leq |\mathcal{O} \times \mathcal{A}|$ . It is clear that this recasting of the original problem might not be very helpful for some Monte-Carlo like tree search methods, however, it might significantly improve the performance of some temporal-difference like algorithms,, e.g. Q-learning [? ], when applied to huge action-spaces.

Note that  $g$  is injective, but it may not be a bijection. There are many sequentialized histories  $\tau \in \check{\mathcal{H}}$  which are not mapped by  $g$ , i.e. there does not exist any history in  $\mathcal{H}$  such that  $\tau = g(h)$ . For such sequentialized histories we define  $g^{-1}(\tau) := \perp$ , which formally allows us to talk about  $g^{-1}$  without worrying about it being undefined on some arguments. The choice of this definition is not important. As a matter of fact, there is no particular significance of the symbol  $\perp$ . What makes this choice insignificant is the fact that the environment does not react until the agent has taken  $d$   $\mathcal{B}$ -ary actions. Some histories not covered by  $g$  are such “partial” sequentialized histories where the actual environment does not react. Note that the rewards of the sequentialized setup are zero ( $r_\perp := 0$ ) unless the sequentialized history length is a multiple of  $d$ , i.e. a “complete” sequentialized history. See [Figure 4](#) for an example sequentialized/binarized setup for  $\mathcal{B} = \mathbb{B}$  and  $d = 2$ .

Any agent which interacts with the environment through this sequentialized scheme would effectively experience the following sequentialized environment.

**Definition 3.2** (Sequentialized environment). For any  $\mathcal{B}$ -ary action  $x \in \mathcal{B}$ , sequentialized history  $\tau \in \check{\mathcal{H}}$ , and any partial extension  $\overline{\mathbf{x}\mathbf{o}\mathbf{r}_\perp}_{\perp < i}$  for  $i \leq d$  the probability of receiving  $o'$  and

<sup>7</sup>The initial history  $e \in \mathcal{O} \times \mathcal{R}$  is similar to the initial state in standard RL. It is dispatched by environment without any input at the start.

<sup>8</sup>This assumption is not much of a restriction, if  $r_\perp \notin \mathcal{R}$  then we can extend the reward space by  $\mathcal{R} \cup \{r_\perp\}$ .

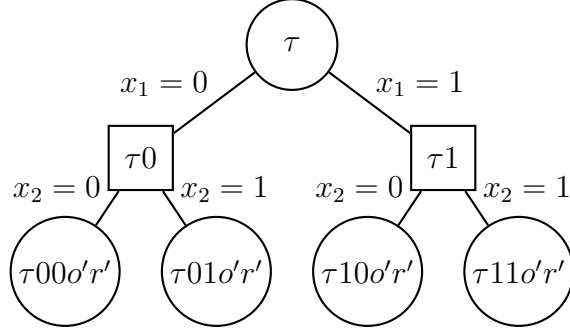


Figure 4: A simple sequentialization/binarization example in a deterministic history-based process. The  $\mathcal{B}$ -ary/binary decisions are on the edges. For brevity, we do not represent  $o_\perp$  and  $r_\perp$  in the figure. For example, it should be apparent that  $\tau 1 o_\perp r_\perp \equiv \tau 1$ . The circles represent complete histories while the squares indicate partial histories.

$r'$  as the next observation and reward is as follows:

$$\check{P}(o'r'|\tau \overline{\mathbf{x} o r}_\perp <_i x) := \begin{cases} P(o'r'|ha) & \text{if } \tau \overline{\mathbf{x} o r}_\perp <_i x o'r' = g(hao'r') \\ 1 & \text{if } o'r' = o r_\perp \\ & \text{and } g^{-1}(\tau \overline{\mathbf{x} o r}_\perp <_i x o'r') = \perp \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where  $P$  is the actual environment.

As highlighted before, the history  $h$  can not be empty, so the above definition is well-defined.

The next step is to define the (action) value functions for this sequentialized agent-environment interaction. Let  $\check{\Pi}$  be a policy such that  $\check{\Pi} : \check{\mathcal{H}} \rightarrow \Delta(\mathcal{B})$ . Then, we define the (action) value functions similar to the original agent-environment interaction case. For any  $\tau \in \check{\mathcal{H}}$  and  $x \in \mathcal{B}$ , the action-value function is defined as

$$\check{Q}^{\check{\Pi}}(\tau, x) := \sum_{o'r'} \check{P}(o'r'|\tau x) \left( r' + \lambda \check{V}^{\check{\Pi}}(\tau x o'r') \right) \quad (8)$$

where  $\check{V}^{\check{\Pi}}(\tau) := \sum_{x \in \mathcal{B}} \check{Q}^{\check{\Pi}}(\tau, x) \check{\Pi}(x|\tau)$  and  $\lambda$  is the discount factor of this sequentialized problem. Similar to the original optimal (action) value functions,  $\check{Q}^*$  and  $\check{V}^*$  denote the optimal (action) value functions of the sequentialized problem. The discount factor  $\lambda$  plays an important role in trading off the size of the action-space with the planning horizon. Recall that the size of the original action-space is  $|\mathcal{A}| = |\mathcal{B}|^d$ . Therefore, if the agent has to make  $d$   $\mathcal{B}$ -ary decisions for each original action the discount factor after  $d$   $\mathcal{B}$ -ary actions should be  $\gamma$ , i.e.  $\lambda^d = \gamma$ . This implies that  $\lambda = \gamma^{1/d} < 1$  as  $\gamma < 1$  and  $d < \infty$ .

This completes the problem setup. We have defined an agent  $\check{\Pi}$  which only makes  $\mathcal{B}$ -ary decisions and reacts to sequentialized histories, see Figure 3. As expected, the set of sequentialized histories  $\check{\mathcal{H}}$  is blown out in comparison with  $\mathcal{H}$ . However, in Section 5, we show that, under certain non-Markovian abstractions of either  $\mathcal{H}$  or  $\check{\mathcal{H}}$ , this expansion is not “harmful”.

## 4 Sequentialized Processes and Values

In this section we formally define the sequentialized process and related value functions. But first we need a couple of important quantities to state our main results. For any  $\mathcal{B}$ -ary vector  $\mathbf{x} \in \mathcal{B}^i$  where  $i \leq d$ , we define  $\mathcal{A}(\mathbf{x}) := \{a \in \mathcal{A} : \mathbf{x} \sqsubseteq C(a)\}$  a *restricted* set of actions. Moreover, for any history  $h$ , an action-value function maximizer on this restricted set is defined as  $\Pi^*(h, \mathbf{x}) \in \arg \max_{a \in \mathcal{A}(\mathbf{x})} Q^*(h, a)$ .

We start off the section by noting an important relationship between the sequentialized process and the original process.

**Proposition 4.1** (Sequentialized Process). *For any  $o' \in \mathcal{O}, r' \in \mathcal{R}, h \in \mathcal{H}$  and  $D(\mathbf{x}) =: a \in \mathcal{A}$ , the following relationship holds between  $\check{P}$  and  $P$ :*

$$\check{P}(o'r'|g(h)\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < d}\mathbf{x}_d) = P(o'r'|ha) \quad (9)$$

*Proof.* The proof trivially follows from Definition 3.2 by evaluating the definition at  $i = d$  with  $D(\mathbf{x}_{< d}\mathbf{x}_d) = a$ .  $\square$

When the original process is an MDP then there exists a sequentialization scheme such that the sequentialized process is also Markovian.

**Theorem 4.2** (Sequentialization preserves Markov property). *If  $P$  is an MDP over  $\mathcal{O}$ , and the observations from the  $\mathcal{B}$ -ary mock are  $\tilde{\mathcal{O}} := \mathcal{O} \times \cup_{i=0}^{d-1} \mathcal{B}^i$ , then  $\check{P}$  is also an MDP over  $\tilde{\mathcal{O}}$ .*

*Proof.* In the case of augmenting the observation space, the definition of  $\check{P}$  becomes slightly more verbose than Definition 3.2 as  $o_{\perp}$  is different for each partial history as defined in Equation (6).

$$\check{P}(\tilde{o}'r'|\tau\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < i}\mathbf{x}) := \begin{cases} P(o'r'|ha) & \text{if } \tau\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < i}\mathbf{x}\tilde{o}'r' = g(hao'r') \\ 1 & \text{if } \tilde{o}'r' = o\mathbf{x}_{< i}\mathbf{x}r_{\perp} \\ & \text{and } g^{-1}(\tau\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < i}\mathbf{x}\tilde{o}'r') = \perp \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

for any  $i \leq d, \tilde{o}, \tilde{o}' \in \tilde{\mathcal{O}}$ , and  $o \in \mathcal{O}$  is the most recent observation in  $h$ . At any  $h$  the sufficient information is  $o$ , so  $P(o'r'|ha) \equiv P(o'r'|oa)$ . Therefore, from the above (expanded) definition of  $\check{P}$ , it is clear that:

$$\check{P}(\tilde{o}'r'|\tau\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < i}\mathbf{x}) \equiv \check{P}(\tilde{o}'r'|o\mathbf{x}_{< i}\mathbf{x}) = \check{P}(\tilde{o}'r'|\tilde{o}\mathbf{x})$$

hence proves the proposition.  $\square$

The following proposition proves that the action-values of the “partial” histories of the sequentialized problem are related. This fact later helps us to show that these action-value functions respect the Q-uniform structure of the original environment.

**Proposition 4.3** ( $\check{Q}^*$  max-relationship). *For any sequentialized history  $\tau \in \mathcal{H}$  such that  $g^{-1}(\tau) \in \mathcal{H}$ , the following holds*

$$\max_{\mathbf{x} \in \mathcal{B}} \check{Q}^*(\tau, \mathbf{x}) = \lambda^{d-1} \max_{\mathbf{x} \in \mathcal{B}^d} \check{Q}^*(\tau\overline{\mathbf{x}\mathbf{o}\mathbf{r}}_{\perp < d}, \mathbf{x}_d) \quad (11)$$

*Proof.* The proof is straight forward. We successively apply the definition of  $\check{Q}^*$ .

$$\begin{aligned}
\max_{x_1 \in \mathcal{B}} \check{Q}^*(\tau, x_1) &= \max_{x_1 \in \mathcal{B}} \sum_{o'r'} \check{P}(o'r'|\tau x_1) \\
&\quad \left( r' + \lambda \max_{x_2 \in \mathcal{B}} \check{Q}^*(\tau x_1 o'r', x_2) \right) \\
&\stackrel{(a)}{=} \lambda \max_{x_1 \in \mathcal{B}} \max_{x_2 \in \mathcal{B}} \check{Q}^*(\tau x_1 o'r_\perp, x_2) \\
&\quad \vdots \text{ (continue unrolling for } d-1 \text{ steps)} \\
&= \lambda^{d-1} \max_{\mathbf{x} \in \mathcal{B}^d} \check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d}, \mathbf{x}_d) \tag{12}
\end{aligned}$$

where (a) follows from the definition of  $\check{P}$  and the fact that  $r' = r_\perp = 0$  when  $\check{P} \neq 0$ .  $\square$

Now, using [Proposition 4.3](#) we can prove a relationship between the action-value functions of the actual environment and the sequentialized environment.

**Lemma 4.4** ( $\check{Q}^*$   $\mathbf{x}$ -relationship). *For any history  $h$  with the corresponding sequentialized history  $\tau = g(h)$  and  $\mathcal{B}$ -ary decision vector  $\mathbf{x} \in \mathcal{B}^d$ , the following holds for any  $i \leq d$ .*

$$\check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < i}, \mathbf{x}_i) = \gamma^{\frac{d-i}{d}} \check{Q}^*(h, \Pi^*(h, \mathbf{x}_{\leq i}))$$

*Proof.* Before we prove the general result, we show that the result holds for  $i = d$ , i.e. the sequentialized problem has same optimal action-values at the “real” decision steps. Note that  $\Pi^*(h, \mathbf{x}_{\leq d}) = D(\mathbf{x})$ . Let  $\mathbf{x} := C(a)$  and  $\tau := g(h)$ . Using the fact that  $r_\perp = \text{constant} = 0$ , we get

$$\begin{aligned}
f_{r_\perp}(h, a) &:= \check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d}, \mathbf{x}_d) \\
&\stackrel{(a)}{=} \sum_{o'r'} \check{P}(o'r'|\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d} \mathbf{x}_d) \left( r' + \lambda \max_{x'} \check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d} \mathbf{x}_d o'r', x') \right) \\
&\stackrel{(b)}{=} \sum_{o'r'} P(o'r'|ha) \left( r' + \lambda \max_{x'} \check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d} \mathbf{x}_d o'r', x') \right) \\
&\stackrel{(c)}{=} \sum_{o'r'} P(o'r'|ha) \left( r' + \lambda^d \max_{\mathbf{x}' \in \mathcal{B}^d} \check{Q}^*(\tau \overline{\mathbf{x} o \mathbf{r}}_{\perp < d} \mathbf{x}_d o'r' \overline{\mathbf{x} o \mathbf{r}}_{\perp < d}', \mathbf{x}'_d) \right) \\
&\stackrel{(d)}{=} \sum_{o'r'} P(o'r'|ha) \left( r' + \gamma \max_{a' \in \mathcal{A}} f_{r_\perp}(ha o'r', a') \right) \tag{13}
\end{aligned}$$

where (a) is just [Equation \(8\)](#) with the optimal policy, (b) follows by [Proposition 4.1](#), (c) is given by [Proposition 4.3](#), (d) is true by rearranging the argument, the definition of  $f_{r_\perp}$  and by using the relation  $\lambda^d = \gamma$ . Note that [Equation \(13\)](#) is the OBE of the original problem, see [Equation \(3\)](#). The solution of the OBE is unique [? ], hence  $f_{r_\perp}$  is indeed  $\check{Q}^*$ .

Having proved the claim for  $i = d$ , we show that it also holds for any  $i < d$ .

$$\begin{aligned}
\check{Q}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i}, \mathbf{x}_i) &\stackrel{(a)}{=} \sum_{o' r'} \check{P}(o' r' | \tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i} \mathbf{x}_i) \left( r' + \lambda \max_{x_{i+1}} \check{Q}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i} \mathbf{x}_i o' r', x_{i+1}) \right) \\
&\stackrel{(b)}{=} \lambda \max_{x_{i+1}} \check{Q}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i} \mathbf{x}_i o r_{\perp}, x_{i+1}) \\
&\quad \vdots \text{ (continue unrolling for } d - i - 1 \text{ steps)} \\
&= \lambda^{d-i} \max_{x_{i+1}} \dots \max_{x_d} \check{Q}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i} \mathbf{x}_i o r_{\perp} x_{i+1} o r_{\perp} \dots x_{d-1} o r_{\perp}, x_d) \\
&\stackrel{(c)}{=} \lambda^{d-i} \max_{a \in \mathcal{A}(\mathbf{x}_{\leq i})} Q^*(h, a)
\end{aligned} \tag{14}$$

where, again (a) is Equation (8) with the optimal policy, (b) follows from the definition of  $\check{P}$  and  $r_{\perp} = 0$ , and (c) is true from the fact that the claim holds for  $i = d$  and the maximum is over the restrictive set of actions.  $\square$

What we have proven so far is that the sequentialization scheme produces action-value functions which (at the “partial” histories) are rescaled versions of the original action-value function. They agree with the original  $Q^*$  at the decision points (at the “complete” histories) where the sequentialized policy  $\check{\Pi}$  completes an action code.

We also show that a similar relationship as proved in Lemma 4.4 holds for a fixed policy  $\check{\Pi}$ . However, we use a different proof method for the following lemma. Note that  $\check{\Pi}$  induces a policy  $\bar{\Pi}$  on the original environment, which can trivially be expressed as follows:

$$\bar{\Pi}(a|h) := \prod_{i=1}^d \check{\Pi}(\mathbf{x}_i | \tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i}) =: \check{\Pi}(\mathbf{x} | \tau) \tag{15}$$

for any  $a = D(\mathbf{x})$  and  $\tau = g(h)$ .

**Lemma 4.5** ( $\check{Q}^{\check{\Pi}}$   $\mathbf{x}$ -relationship). *For any arbitrary policy  $\check{\Pi}$  the following relationship is true:*

$$\check{Q}^{\check{\Pi}}(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d}, \mathbf{x}_d) = Q^{\bar{\Pi}}(h, D(\mathbf{x})) \tag{16}$$

for any history  $\tau = g(h)$  and  $\mathbf{x} \in \mathcal{B}^d$ .

*Proof.* Before we prove the main result of the lemma, we show that the following relationship holds for the value-functions of the sequentialized and the original environment:

$$V^{\check{\Pi}}(\tau) = \lambda^{d-1} V^{\bar{\Pi}}(h) \tag{17}$$

for any  $\tau = g(h)$ . We use a different argument than Lemma 4.4 to prove the above statement. Lets imagine the sequentialized environment is at the history  $\tau = g(h)$ . The agent starts to follow the policy  $\check{\Pi}$ . The following is the (expected-reward, discount-factor) sequence it generates from this history.

$$\begin{aligned}
&(0, \lambda^0), (0, \lambda^1), \dots, (0, \lambda^{d-2}), (\bar{r}, \lambda^{d-1}), \\
&(0, \lambda^d), (0, \lambda^{d+1}), \dots, (0, \lambda^{2d-2}), (\bar{r}', \lambda^{2d-1}), \\
&(0, \lambda^{2d}), \dots
\end{aligned}$$

where  $\bar{r}$  is the expected reward. The sum of the reward part of the above sequence returns  $\check{V}^{\check{\Pi}}(\tau)$ . Now, if we re-scale the discount part of the above sequence by  $\lambda^{d-1}$  we get  $V^{\bar{\Pi}}(h)$  as the sum of the reward part.

$$\begin{aligned} & (0, \lambda^{1-d}), (0, \lambda^{2-d}), \dots, (0, \lambda^{-1}), (\bar{r}, \lambda^0), \\ & (0, \lambda^1), (0, \lambda^2), \dots, (0, \lambda^{d-1}), (\bar{r}', \lambda^d), \\ & (0, \lambda^{d+1}), \dots \end{aligned}$$

which proves [Equation \(17\)](#) when  $\lambda^d = \gamma$ . Now, let  $a := D(\mathbf{x})$ .

$$\begin{aligned} Q^{\check{\Pi}}(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d}, \mathbf{x}_d) &= \sum_{o' r'} \check{P}(o' r' | \tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d} \mathbf{x}_d) \left( r' + \lambda V^{\check{\Pi}}(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d} \mathbf{x}_d o' r') \right) \\ &\stackrel{(a)}{=} \sum_{o' r'} P(o' r' | h a) \left( r' + \lambda V^{\check{\Pi}}(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d} \mathbf{x}_d o' r') \right) \\ &\stackrel{(17)}{=} \sum_{o' r'} P(o' r' | h a) \left( r' + \lambda^d V^{\bar{\Pi}}(h a o' r') \right) \\ &= \sum_{o' r'} P(o' r' | h a) \left( r' + \gamma V^{\bar{\Pi}}(h a o' r') \right) = Q^{\bar{\Pi}}(h, D(\mathbf{x})) \end{aligned}$$

where (a) is due to [Proposition 4.1](#). □

The following theorem proves the usefulness of our sequentialization framework. We show that the optimal policy of the sequentialized environment is also optimal in the original environment when it is lifted back using the decoding function  $D$ .

**Theorem 4.6** (Sequentialization preserves  $\varepsilon$ -optimality). *Any  $\lambda^{d-1}\varepsilon$ -optimal policy of the sequentialized environment is  $\varepsilon$ -optimal in the original environment.*

*Proof.* Let  $\check{\Pi}$  be an  $\varepsilon'$ -optimal policy of the sequentialized environment, where  $\varepsilon' := \lambda^{d-1}\varepsilon$ . It implies the following:

$$\check{V}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i}) - \check{V}^{\check{\Pi}}(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < i}) \leq \varepsilon' \quad (18)$$

for any complete sequentialized history  $\tau = g(h)$  and  $\mathbf{x} \in \mathcal{B}^{i-1}$  where  $i \leq d$ . Especially, we are interested in the case when  $i = 1$ , i.e. values at the complete histories.

$$\check{V}^*(\tau) - \check{V}^{\check{\Pi}}(\tau) \leq \varepsilon' \quad (19)$$

With simple algebra, we can show that the following relationship holds for the optimal policies of the sequentialized and original processes:

$$\begin{aligned} \check{V}^*(\tau) &\stackrel{(a)}{=} \max_x \check{Q}^*(\tau, x) \\ &\stackrel{(b)}{=} \lambda^{d-1} \max_{\mathbf{x} \in \mathcal{B}^d} \check{Q}^*(\tau \overline{\mathbf{x} \mathbf{o} \mathbf{r}}_{\perp < d}, \mathbf{x}_d) \\ &\stackrel{(c)}{=} \lambda^{d-1} \max_{\mathbf{x} \in \mathcal{B}^d} Q^*(h, D(\mathbf{x})) = \lambda^{d-1} V^*(h) \end{aligned} \quad (20)$$

where (a) is the definition of the value function, (b) holds due to [Proposition 4.3](#), and (c) is true by applying [Lemma 4.4](#) for  $i = d$ .

Now, by simply using Equation (17) and Equation (20), we can prove the claim.

$$V^*(h) - V^{\bar{\Pi}}(h) \stackrel{(a)}{=} \lambda^{1-d} \left( \check{V}^*(\tau) - \check{V}^{\check{\Pi}}(\tau) \right) \stackrel{(19)}{\leq} \varepsilon \quad (21)$$

for any  $\tau = g(h)$ , where (a) is due to Equation (17) and Equation (20).  $\square$

We are done formally defining the setup. In the next section we put everything together under the context of ESA to establish the validity of our sequentialization setup.

## 5 Extreme State Aggregation

In the previous sections, we formalized the GRL problem with a sequentialized action-space. A GRL agent keeps the history of its interaction to decide the next action. The history grows with time but even worse is that, without more assumptions and/or abstractions, no history ever repeats [? ]. This is a unique characteristic of the history-based setup which sets it apart from the standard RL [? ]. It enables the GRL framework to cover from the extreme case of unique histories to the most restrictive scenarios of bandits. However, without abstractions, a GRL agent which assumes every history is unique is more of a theoretical artifact than a realizable algorithm. It is critical to note that our sequentialization scheme results in just like any other GRL agent. It also requires an abstraction map (or further structural assumptions) to provide an implementable algorithm. Usually, one starts by assuming some structure on the history set(s). After reviewing some, we will argue against all of them.

On one extreme we have unique histories and on the other end, typically, the environment distribution is assumed to be Markovian, i.e. for any  $h$  and  $a$ ,  $P(o'r'|ha) \equiv P(o'r'|oa)$  for all  $o'r'$  where  $o$  is the most recent observation in  $h$  [? ]. This means that histories with the same most recent observation are members of the same class (or state). This assumption provides a lot of structure on  $\mathcal{H}$ . The value functions become functions of the most recent observations. Note that [? ] defines the Markovian assumption directly on histories, which is a bit weaker than what we have stated above.

Unfortunately, the Markovian assumption is too strong to be used in many real-world problems. We do not interact with the world based on just our recent observations. As general agents, we keep “relevant” historical events in memory to plan better in the future, sometimes optimally. Apart from some “toy” examples and (well-defined) games [? ? ? ], this assumption demands too much structure on the history space. So, what other assumption can we make? We can keep the Markovian structure but can weaken the assumption, significantly, by assuming that the agent is not able to observe the state directly. The agent may require a sufficiently long history of interaction to discern the *hidden* Markovian state of the environment [? ]. The class of problems this assumption models is known as partially observable Markov decision problems (POMDP). Almost all problems we care about can be modeled as POMDPs. However, POMDP solution methods are very demanding and the optimal behavior is not guaranteed to be learnable in general [? ]. We do not address this non-MDP class any further.

We focus on other important quantities in GRL formulation, e.g.  $\Pi^*$ ,  $V^*$ , and  $Q^*$ , and make no direction assumption on  $P$ . This has been a subject of many works; [? ? ] considered a unified abstraction framework by mapping states similar in value, [? ] subsumed the previous work by considering the GRL setup, and [? ] extended the work to state-action abstractions. The value functions provide natural criteria to group histories. The resultant structure can be non-Markovian. Such non-MDP abstractions have many benefits over Markovian reductions.

The resultant state-space ( $\cong$  the set of groups of histories) can be significantly smaller with these abstractions than the Markovian counterparts. One advantage of using such abstractions, as compared to POMDPs, is the guarantee of the optimal behavior being a function of states, which helps the learning in many problems that were traditionally not considered learnable [? ].

However, the most remarkable aspect of such non-MDP abstractions is that there may exist an upper bound on the required number of states *uniformly* for any problem, as it is the case in ESA [? ]. The idea is to group histories together which have *similar* optimal action-values  $Q^*$ . Since  $Q^*$  is a bounded real function (which is the case as  $\mathcal{R}$  is bounded), we can potentially upper bound the required number of states by lumping together histories by discretization of the action-value function. In this work, we are primarily interested in non-MDP abstractions of the following type.

**Definition 5.1** ( $\varepsilon$ -Q-uniform abstraction). *An abstraction function  $\phi : \mathcal{H} \rightarrow \mathcal{S}$  is an  $\varepsilon$ -Q-uniform abstraction if for any  $h, \dot{h} \in \mathcal{H}$  and all  $a \in \mathcal{A}$  we have*

$$\left( \phi(h) = \phi(\dot{h}) \right) \implies \left| Q^*(h, a) - Q^*(\dot{h}, a) \right| \leq \varepsilon$$

where  $\mathcal{S}$  is the set of states<sup>9</sup> of the abstraction.

In ESA, the agent’s policy is (constrained to be only) a function of the states. Although these states do not exhibit Markovian dynamics, the agent can “pretend” that the abstract process is Markovian. This structure provides a surrogate-MDP whose optimal policy is  $\varepsilon$ -optimal in the original environment.

The  $\varepsilon$ -Q-uniform, non-MDP abstractions lead to the following important result due to [? ]. We only state the result without a proof for the closure of exposition, see [? ] for more details about ESA and proofs.

In the following theorems we assume that the rewards are bounded in the unit interval, i.e.  $\mathcal{R} \subseteq [0, 1]$ . This is done for brevity, and it is not a necessary condition. The rescaling of the rewards does not affect the decision-making process in (G)RL. In general, let the range of the rewards be  $R := \max \mathcal{R} - \min \mathcal{R}$ . Then, the scalars in the nominators of Theorems 5.2 and 5.3 are replaced by  $2R$  and  $4R^2$  respectively.

**Theorem 5.2** (ESA [? , Theorem 11]). *For every environment  $P$  there exists a reduction  $\phi$  and a surrogate-MDP whose optimal policy<sup>10</sup> is an  $\varepsilon$ -optimal policy for the environment. The size of the surrogate-MDP is bounded (uniformly for any  $P$ ) by<sup>11</sup>*

$$|\mathcal{S}| \leq \left( \frac{2}{\varepsilon(1 - \gamma)^3} \right)^{|\mathcal{A}|}$$

This is a powerful result, but it suffers from the exponential dependence on the action-space size. We now put our action sequentialization framework to work and dramatically improve this dependency from exponential to only a logarithmic dependency in  $|\mathcal{A}|$ .

So far, we have considered an arbitrary  $\mathcal{B}$ -ary decision set to sequentialize the action-space. However, in the following theorem we go to the extreme case of sequentializing the

<sup>9</sup>We consider a finite abstract set of states, but the underlying set of states ( $\cong$  history-space) is (allowed to be) infinite. Note also that we consider *approximate* Q-uniform aggregations which result into a finite abstract state.

<sup>10</sup>See [? ] of how to learn this policy, the surrogate-MDP,  $Q^*$ , and  $\phi$ .

<sup>11</sup>The 2 instead of a 3 in the original theorem is a trivial improvement by removing the grid point at 0 in the construction.

action-space to binary decisions ( $\mathcal{B} = \mathbb{B}$ ) to squeeze out the maximum improvement possible through the framework.

**Theorem 5.3** (Binary ESA). *For every environment there exists an abstraction and a corresponding surrogate-MDP for its binarized version ( $\mathcal{B} = \mathbb{B}$ ) whose optimal policy is  $\varepsilon$ -optimal for the true environment. The size of the surrogate-MDP is uniformly bounded for every environment as*

$$|\mathcal{S}| \leq \frac{4[1 - \gamma + \log_2 |\mathcal{A}|]^6}{\gamma^2 \varepsilon^2 (1 - \gamma)^6}$$

*Proof.* Consider the agent that is interacting with the sequentialized/binarized environment  $\check{P}$ . By [Theorem 4.6](#), we know that a near-optimal policy of this sequentialized environment is also near-optimal in the original environment. Now, if we use ESA on the binarized problem and get an  $\varepsilon'$ -optimal policy through the surrogate-MDP by [Theorem 5.2](#), we are sure to be  $\varepsilon$ -optimal in the original environment  $P$  as explained above. Additionally, the size of the state-space is bounded as

$$|\mathcal{S}| \stackrel{\text{Theorem 5.2}}{\leq} \left( \frac{2}{\varepsilon'(1 - \lambda)^3} \right)^2 = \frac{4}{\varepsilon'^2 (1 - \lambda)^6} \quad (22)$$

where  $\lambda$  is the discount factor of the sequentialized problem. Next, we upper bound [Equation \(22\)](#) by using the fact that  $\lambda^d = \gamma$ . Let  $\delta := 1 - \gamma < 1$ . So,

$$\begin{aligned} 1 - \lambda &= 1 - (1 - \delta)^{1/d} = 1 - e^{\frac{\ln(1 - \delta)}{d}} \\ &\stackrel{(a)}{\geq} 1 - \frac{1}{1 - \ln(1 - \delta)/d} \stackrel{(b)}{\geq} 1 - \frac{1}{1 + \delta/d} \\ &= \frac{\delta}{d + \delta} = \frac{1 - \gamma}{d + 1 - \gamma} \end{aligned} \quad (23)$$

where (a) holds due to  $\frac{1}{e^{-\alpha}} \leq \frac{1}{1 - \alpha}$ , (b) is true by using the fact that  $\delta < 1$ , hence  $\ln(1 - \delta) \leq -\delta$ . Therefore, using [Equation \(22\)](#), [Equation \(23\)](#), and  $\varepsilon' = \lambda^{d-1} \varepsilon \geq \lambda^d \varepsilon = \gamma \varepsilon$  we get,

$$|\mathcal{S}| \leq \frac{4}{\varepsilon'^2 (1 - \lambda)^6} \leq \frac{4(1 - \gamma + d)^6}{\gamma^2 \varepsilon^2 (1 - \gamma)^6} \quad (24)$$

which proves the claim.  $\square$

Superficially, it might seem that we have simply replaced the original discount factor with a larger value. But, it is not the case. If we simply scaled the discount factor (without sequentializing the actions) then the resulting bound would indeed deteriorate, see [Theorem 5.2](#), but on the contrary, with sequentialization/binarization and our analysis the bound (dramatically) improves.

Usually in RL the discount factor  $\gamma$  is close to 1. In that case, the bound in [Theorem 5.3](#) can be tightened further as:

$$|\mathcal{S}| \lesssim \frac{4[\log_2 |\mathcal{A}|]^6}{\varepsilon^2 (1 - \gamma)^6} \quad (25)$$

which agrees with the bound in [Theorem 5.2](#) for the case when  $|\mathcal{A}| = 2$ , i.e. when the original problem already has a binary action-space.

## 6 Conclusion & Outlook

This work contributes to the study of the GRL problem. We have provided a reduction to handle large state and action spaces by sequentializing the decision-making process. This helped us improve the upper bound on the number of states in ESA from an exponential dependency in  $|\mathcal{A}|$  to logarithmic. The gain is *double exponential* in terms of the action-space dependence at no other cost. Our result carries a broader impact on the implementation of *general* RL agents<sup>12</sup>. The required storage for such agents, which have access to a non-MDP, approximate Q-uniform abstraction, can be reasonably bounded which only scales logarithmically in the size of the action-space.

We conclude the paper with some future research directions. This work analyses the case when the agent has a fixed aggregation map. [?] provides an outline for a learning algorithm to learn such abstractions which can be combined with our sequentialization framework. Another direction, which we also did not touch in this work, is to explore the connection, if any, between the surrogate-MDPs of a map on the original environment, and its extension on the sequentialized problem. By lifting the small binary ESA map, say  $\psi$ , back to  $\mathcal{H}$ , one obtains a small map directly on  $\mathcal{H}$ , say  $\phi$ . While  $\psi$  used sequentialization/binarization for the construction of  $\phi$ , the map  $\phi$  can be used without further referencing to sequentialization. This suggests that a bound logarithmic in  $|\mathcal{A}|$  should be possible without a detour through the sequentialization. This deserves further investigation.

We sequentialize the action-space through an *arbitrary* coding scheme  $C$ , so the main result does not depend on this choice. Sometimes, it is possible that the action-space may allow natural sequentialization, e.g. in a video game controller the macro action might be a binary vector where the first bit might represent the left/right direction, the second bit indicates up/down, and so on. The exact nature of these binary decisions depends on the domain which is reflected by the choice of encoding  $C$ . Sequentialization was our path to double-exponentially improve that bound. Whether there are more direct/natural aggregations with the same bound is an open problem. Moreover, if the agent is learning an abstraction through interaction, the choice of these functions may become critical. This paper focused on rigorously formalizing and proving the main improvement result. One can also try to empirically show the effectiveness of our improved upper bound. To do this, we need a problem domain where ESA requires more states than the sequentialized/binarized version of it. But a point of caution is that the upper bound still scales badly in terms of  $\gamma$  and  $\varepsilon$ . Any reasonable value of these parameters would imply a huge upper bound. Even with Markovian abstractions, a cubic dependency on the discount factor is the best achievable. We considered a general underlying process and non-Markovian abstractions, and dramatically improved the previously best bound  $(1 - \gamma)^{-3|\mathcal{A}|}$  to  $(1 - \gamma)^{-3 \cdot 2}$ . Indeed it would be interesting to see whether this can be further improved to the optimal  $(1 - \gamma)^{-3}$  rate.

**Acknowledgements.** This work has been supported by Australian Research Council grant DP150104590. Thanks to the anonymous reviewers for their feedback and to Andrs Gyrgy who pointed out that the sequentialized/binarized process in Figure 2 preserves the Markov property, which encouraged us to also consider the Markov case.

Esse fugiat asperiores veniam tenetur similique praesentium facere, mollitia repellat dolorem maiores atque tenetur porro, ab ea voluptas eius corrupti atque nesciunt ex, animi atque eum officiis suscipit voluptatum molestiae, mollitia in praesentium doloribus modi hic molestias debitis inventore sint perferendis. Iure voluptas minus eligendi quibusdam incidunt porro quia iste corporis voluptates, in recusandae veniam autem odio perspiciatis consequuntur officia voluptatem, earum recusandae harum eius officia?

<sup>12</sup>The general (or strong) agents are designed to work with a wide range of environments [? ].